RAJALAKSHMI INSTITUTE OF TECHNOLOGY

(An Autonomous Institution, Affiliated to Anna University, Chennai)

DEPARTMENT OF CSE (ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

ACADEMIC YEAR 2025 - 2026

SEMESTER IV

AL23421 - MACHINE LEARNING LABORATORY

MINI PROJECT REPORT

REGISTER NUMBER	2117240030016
NAME	BARATH S
PROJECT TITLE	**SOIL NUTRIENT BASED CROP RECOMMENDATION USING DECISION TREE ALGORITHM**
DATE OF SUBMISSION	
FACULTY IN-CHARGE	**Mrs. M. Divya**

Signature of Faculty In-charge **Signature of HoD**

Internal Examiner **External Examiner**

INTRODUCTION

Agriculture is increasingly being transformed by technology, especially through Machine Learning and Deep Learning techniques. One of the major challenges faced by farmers is selecting the right crop for cultivation without having sufficient knowledge about soil characteristics and environmental conditions.

- **Deep Learning (MLP Neural Network)** → for soil type classification using images
- **Machine Learning (Decision Tree Algorithm)** → for crop prediction

The system is deployed as a **web-based application**, allowing users to upload soil images and input environmental parameters (temperature, humidity, rainfall). The system then processes the inputs and provides an accurate crop recommendation.

Traditional crop selection methods rely on experience or external consultation, which may not always be accurate. Many farmers do not have access to soil testing facilities, making it difficult to determine soil properties such as texture and composition.

This project introduces a **smart crop recommendation system** that removes the need for prior knowledge. The user simply uploads an image of the soil, and the system automatically:

The system aims to bridge the gap between agriculture and technology, enabling farmers to make informed decisions easily.

GOAL

The primary goal of this project is to develop a user-friendly Web Application that accurately recommends crops based on soil imagery and real-time environmental data.

Possibilities and milestones include:

- Processing images directly from the user to classify the soil type automatically using an MLP Neural Network.
- Utilizing a Decision Tree to compute the final crop recommendation based on Rainfall, Temperature, Humidity, and the inferred Soil Type.
- Displaying real-time analytical dashboards of soil and crop data utilizing a SQLite database backend.
- Expansion to smart agriculture systems

THEORETICAL BACKGROUND

1. Deep Learning for Image Analysis (MLP)

The system uses a **Multi-Layer Perceptron (MLP)** to classify soil images.

Equation:

$$z = \sum w_i x_i + b$$
$$a = f(z)$$

The model learns patterns such as:

- Soil color
- Texture

---

2. Decision Tree for Crop Prediction

The Decision Tree algorithm uses:

- Soil type (from image)
- Temperature
- Humidity
- Rainfall

Entropy:

$$Entropy(S) = -\sum p_i \log_2(p_i)$$

Information Gain:

$$Gain(S, A) = Entropy(S) - \sum \frac{|S_v|}{|S|} Entropy(S_v)$$

---

3. Justification

A **Decision Tree** is used as the main Machine Learning algorithm. It is a supervised learning method that works by splitting data into smaller subsets based on feature values such as soil type, temperature, humidity, and rainfall. The algorithm selects the best feature for splitting using

measures like **Entropy** and **Information Gain**, which help in reducing uncertainty and improving prediction accuracy. This process continues until a final decision (crop recommendation) is reached at the leaf node.

To handle image input, a **Multi-Layer Perceptron (MLP)** model is used from Deep Learning. It processes the uploaded soil image, extracts patterns such as color and texture, and classifies the soil type.

Thus, the system follows a hybrid approach:

- **Deep Learning (MLP)** → for soil image classification
- **Decision Tree (ML)** → for crop prediction

This combination ensures that both unstructured data (images) and structured data (environmental factors) are effectively utilized for accurate and reliable crop recommendations.

ALGORITHM EXPLANATION WITH EXAMPLE

The system uses a **two-stage algorithm** combining Deep Learning and Machine Learning to recommend crops.

Step 1: Soil Classification (Deep Learning - MLP)

- The user uploads a soil image
- The image is converted into pixel values
- The **MLP model** analyzes color and texture patterns
- It predicts the **soil type** (e.g., Red, Black, Sandy,Clay,Alluvial)

Step 2: Crop Prediction (Decision Tree)

- The predicted soil type is combined with:
  - Temperature
  - Humidity
  - Rainfall
- These inputs are passed into the **Decision Tree algorithm**
- The tree evaluates conditions step-by-step and predicts the most suitable crop

Example

Input:

- Soil Image → Predicted as **Black Soil**
- Temperature = 30°C
- Humidity = 65%
- Rainfall = 110 mm

Decision Process:

1. Check if Rainfall ≤ 120 → Yes
2. Check if Humidity ≤ 70 → Yes
3. Check if Soil Type = Black → Yes

Output:
Recommended Crop: **Maize**
Confidence Level: **97%**

IMPLEMENTATION CODE AND DATASET USED

1. DATASET SPECIFICATIONS

Category	Description
Dataset Name	Synthetic Crop Recommendation Dataset
Domain	Agriculture / Smart Farming
Objective	Multiclass Classification
Data Source	Synthetic (Generated using Python)
Number of Samples	3300 (22 crops × 150 samples each)
Data Type	Tabular
Instance Description	Each row represents environmental conditions and soil type mapped to a crop
Features (Inputs)	Temperature, Humidity, Rainfall, Soil,Type
Feature Types	Numerical (temperature, humidity, rainfall), Categorical (soil_type)
Target Variable	label (crop name)
Number of Classes	22 crops
License	Open (Generated dataset)
Version	V1.0

Instead of using external datasets, this project uses a **custom synthetic dataset** generated using realistic agricultural ranges.

Key Idea:

- Each crop is assigned:
- Suitable temperature range
- Humidity range
- Rainfall range
- Soil types
- Then random values are generated within those ranges.

Implementation Code

1.dataset_generator.py Code

```
import pandas as pd

import numpy as np

import random

def generate_crop_data():

    np.random.seed(42)

    random.seed(42)

    # Define realistic ranges for each crop based on agricultural standards

    # columns: temperature, humidity, rainfall, soil_type, label

    crop_profiles = {

        'rice': {'soil_types': ['alluvial', 'clay'], 'temperature': (20, 27), 'humidity': (80, 85), 'rainfall': (180, 260)},

        'maize': {'soil_types': ['red', 'black', 'alluvial'], 'temperature': (18, 27), 'humidity': (55, 75), 'rainfall': (60, 110)},

        'chickpea': {'soil_types': ['sandy', 'alluvial'], 'temperature': (17, 21), 'humidity': (14, 20), 'rainfall': (65, 95)},

        'kidneybeans': {'soil_types': ['alluvial', 'red'], 'temperature': (15, 25), 'humidity': (18, 24), 'rainfall': (60, 150)},

        'pigeonpeas': {'soil_types': ['alluvial', 'sandy'], 'temperature': (18, 37), 'humidity': (30, 70), 'rainfall': (90, 200)},
```

'mothbeans': {'soil_types': ['sandy', 'red'], 'temperature': (24, 32), 'humidity': (40, 65), 'rainfall': (30, 75)},

'mungbean': {'soil_types': ['alluvial', 'sandy'], 'temperature': (27, 30), 'humidity': (80, 90), 'rainfall': (36, 60)},

'blackgram': {'soil_types': ['alluvial', 'black'], 'temperature': (25, 35), 'humidity': (60, 68), 'rainfall': (60, 75)},

'lentil': {'soil_types': ['alluvial', 'clay'], 'temperature': (18, 30), 'humidity': (60, 69), 'rainfall': (35, 55)},

'pomegranate': {'soil_types': ['red', 'alluvial'], 'temperature': (18, 24), 'humidity': (85, 95), 'rainfall': (100, 115)},

'banana': {'soil_types': ['alluvial', 'clay'], 'temperature': (25, 30), 'humidity': (75, 85), 'rainfall': (90, 120)},

'mango': {'soil_types': ['alluvial', 'red', 'black'], 'temperature': (27, 36), 'humidity': (45, 55), 'rainfall': (85, 105)},

'grapes': {'soil_types': ['sandy', 'alluvial'], 'temperature': (8, 42), 'humidity': (80, 84), 'rainfall': (65, 75)},

'watermelon': {'soil_types': ['sandy', 'alluvial'], 'temperature': (24, 26), 'humidity': (80, 90), 'rainfall': (40, 60)},

'muskmelon': {'soil_types': ['sandy', 'alluvial'], 'temperature': (27, 30), 'humidity': (90, 95), 'rainfall': (20, 30)},

'apple': {'soil_types': ['alluvial', 'red'], 'temperature': (21, 24), 'humidity': (90, 95), 'rainfall': (100, 125)},

'orange': {'soil_types': ['alluvial', 'red'], 'temperature': (10, 35), 'humidity': (90, 95), 'rainfall': (100, 120)},

'papaya': {'soil_types': ['alluvial', 'black'], 'temperature': (23, 44), 'humidity': (90, 95), 'rainfall': (40, 250)},

'coconut': {'soil_types': ['sandy', 'alluvial'], 'temperature': (25, 30), 'humidity': (90, 100), 'rainfall': (130, 230)},

'cotton': {'soil_types': ['black', 'clay'], 'temperature': (22, 26), 'humidity': (75, 85), 'rainfall': (60, 100)},

```
        'jute': {'soil_types': ['alluvial', 'clay'], 'temperature': (23, 27), 'humidity': (70, 90), 'rainfall': (150, 200)},

        'coffee': {'soil_types': ['red', 'alluvial'], 'temperature': (23, 28), 'humidity': (50, 70), 'rainfall': (115, 200)}

    }

    data = []

    # Generate 150 samples per crop

    for label, ranges in crop_profiles.items():

        for _ in range(150):

            row = {

                'temperature': np.random.uniform(ranges['temperature'][0], ranges['temperature'][1]),

                'humidity': np.random.uniform(ranges['humidity'][0], ranges['humidity'][1]),

                'rainfall': np.random.uniform(ranges['rainfall'][0], ranges['rainfall'][1]),

                'soil_type': random.choice(ranges['soil_types']),

                'label': label

            }

            data.append(row)

  df = pd.DataFrame(data)

    # Shuffle the dataset

    df = df.sample(frac=1).reset_index(drop=True)

    # Save to CSV

    df.to_csv('dataset.csv', index=False)

    print("New dataset.csv successfully generated with {} rows.".format(len(df)))

if __name__ == '__main__':
```

```
generate_crop_data()
```

2.model.py Code

```
import os
import sqlite3
import numpy as np
import pickle
import logging
from flask import Flask, request, jsonify, render_template
from PIL import Image
import io
import torch
import torchvision.transforms as transforms
from torchvision.models import mobilenet_v3_small, MobileNet_V3_Small_Weights
app = Flask(__name__)
# Configure logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
DB_FILE = os.path.join(BASE_DIR, 'database.db')
CROP_MODEL_FILE = os.path.join(BASE_DIR, 'crop_pipeline.pkl')
SOIL_MODEL_FILE = os.path.join(BASE_DIR, 'soil_model.pkl')
SOIL_CLASSES_FILE = os.path.join(BASE_DIR, 'soil_classes.pkl')
# Helper to pre-populate DB with 150 items so analytics works instantly
```

```
def seed_database(conn):
    c = conn.cursor()
    c.execute('SELECT COUNT(*) FROM soil_data')
    if c.fetchone()[0] == 0:
        logging.info("Seeding DB with static fallback data for Analytics...")
        from dataset_generator import generate_crop_data
        import pandas as pd
        if not os.path.exists('dataset.csv'):
            generate_crop_data()
        df = pd.read_csv('dataset.csv')
        # Just grab random 150 samples
        df_sample = df.sample(n=150, random_state=42)
        for _, row in df_sample.iterrows():
            c.execute('''
                INSERT INTO soil_data (temperature, humidity, rainfall, soil_type,
recommended_crop)
                VALUES (?, ?, ?, ?, ?)
            ''', (
                float(row['temperature']), float(row['humidity']),
                float(row['rainfall']), str(row['soil_type']),
                str(row['label'])
            ))
        conn.commit()
def init_db():
```

```
try:
    # We will drop the old schema and recreate if N exists
    conn = sqlite3.connect(DB_FILE)
    c = conn.cursor()
    c.execute("PRAGMA table_info(soil_data)")
    columns = [info[1] for info in c.fetchall()]
    if 'N' in columns:
        logging.info("Old DB Schema detected. Dropping old soil_data table.")
        c.execute("DROP TABLE soil_data")
    c.execute('''
        CREATE TABLE IF NOT EXISTS soil_data (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            temperature REAL NOT NULL,
            humidity REAL NOT NULL,
            rainfall REAL NOT NULL,
            soil_type TEXT NOT NULL,
            recommended_crop TEXT
        )
    ''')
    conn.commit()
    seed_database(conn)
    conn.close()
    logging.info("Database initialized.")
```

```
    except Exception as e:
        logging.error(f"DB Init Error: {e}")
with app.app_context():
    init_db()
# Load Models
def load_models():
    crop_pipeline, soil_model, soil_classes = None, None, None
    if os.path.exists(CROP_MODEL_FILE):
        with open(CROP_MODEL_FILE, 'rb') as f:
            crop_pipeline = pickle.load(f)
    if os.path.exists(SOIL_CLASSES_FILE):
        with open(SOIL_CLASSES_FILE, 'rb') as f:
            soil_classes = pickle.load(f)
    if os.path.exists(SOIL_MODEL_FILE):
        with open(SOIL_MODEL_FILE, 'rb') as f:
            soil_model = pickle.load(f)
    return crop_pipeline, soil_model, soil_classes
crop_pipeline, soil_model, soil_classes = load_models()
# Load PyTorch Validation Model
print("Loading PreTrained PyTorch ImageValidator...")
validation_weights = MobileNet_V3_Small_Weights.DEFAULT
validation_model = mobilenet_v3_small(weights=validation_weights)
validation_model.eval()
```

```
validation_preprocess = validation_weights.transforms()

imagenet_categories = validation_weights.meta["categories"]

def is_soil_image(image_bytes):

    """Uses a pre-trained ImageNet model to verify if the uploaded picture is actually
soil/ground/dirt"""

    img = Image.open(io.BytesIO(image_bytes)).convert('RGB')

    batch = validation_preprocess(img).unsqueeze(0)

    with torch.no_grad():

        prediction = validation_model(batch).squeeze(0).softmax(0)

    # Get top 5 predictions

    top5_prob, top5_catid = torch.topk(prediction, 5)

    # Check if any of the top 5 descriptions relate to ground/soil/dirt/mud/sand/field

    valid_keywords = ['soil', 'dirt', 'earth', 'ground', 'sand', 'mud', 'field', 'pot', 'plow', 'tractor']

    for i in range(top5_prob.size(0)):

        category_name = imagenet_categories[top5_catid[i]].lower()

        if any(keyword in category_name for keyword in valid_keywords):

            return True, category_name

    # If we got here, it's likely a face, car, animal, etc.

    top_prediction = imagenet_categories[top5_catid[0]]

    return False, top_prediction


def process_image(img_bytes):

    image = Image.open(io.BytesIO(img_bytes)).convert('RGB')

    image = image.resize((64, 64))
```

```
    img_array = np.array(image) / 255.0

    return img_array.flatten().reshape(1, -1)

@app.route('/')

def index():

    return render_template('index.html')

@app.route('/predict', methods=['POST'])

def predict():

    try:

        if 'image' not in request.files:

            return jsonify({'error': 'No image uploaded for Soil Analysis'}), 400

        file = request.files['image']

        temp = request.form.get('temperature')

        file_bytes = file.read()

        # 0. Pre-Validation

        # is_valid, detected_object = is_soil_image(file_bytes)

        # if not is_valid:

        #     return jsonify({

        #         'error': f'Invalid Image: This looks like a "{detected_object}". Please choose a correct
image of soil for prediction.'

        #     }), 400

        # 1. Decision Tree Classification for Soil Type

        if not soil_model or not soil_classes:

            return jsonify({'error': 'Decision Tree Soil Model not found on the server.'}), 500

        img_array = process_image(file_bytes)
```

```
humidity = request.form.get('humidity')
rainfall = request.form.get('rainfall')
if not temp or not humidity or not rainfall:
    return jsonify({'error': 'Missing environmental inputs'}), 400
temp = float(temp)
humidity = float(humidity)
rainfall = float(rainfall)
# Decision Tree Predict
soil_class_idx = soil_model.predict(img_array)[0]
try:
    probs = soil_model.predict_proba(img_array)[0]
    soil_confidence = float(max(probs)) * 100
except:
    soil_confidence = 85.0
if soil_confidence <= 50.0:
    return jsonify({'error': 'choose a correct image of soil for prediction'}), 400
predicted_soil = soil_classes[soil_class_idx]
# 2. Decision Tree for Crop Recommendation
if not crop_pipeline:
    return jsonify({'error': 'ML crop model not loaded.'}), 500
# Construct input dataframe matching training shape
import pandas as pd
input_df = pd.DataFrame([{
```

```
            'temperature': temp,
            'humidity': humidity,
            'rainfall': rainfall,
            'soil_type': predicted_soil
        }])
        crop_prediction = crop_pipeline.predict(input_df)[0]
        # Calculate confidence if method exists
        try:
            crop_probs = crop_pipeline.predict_proba(input_df)[0]
            crop_confidence = float(max(crop_probs)) * 100
        except:
            crop_confidence = 85.0 # fallback
        logging.info(f"Soil: {predicted_soil} ({soil_confidence:.1f}%), Crop: {crop_prediction}")
        return jsonify({
            'soil_type': predicted_soil,
            'soil_confidence': f"{soil_confidence:.2f}%",
            'recommended_crop': str(crop_prediction).capitalize(),
            'crop_confidence': f"{crop_confidence:.2f}%"
        })
    except Exception as e:
        logging.error(f"Prediction error: {str(e)}")
        return jsonify({'error': str(e)}), 500
@app.route('/soil', methods=['GET'])
```

```
def get_soil_data():
    try:
        conn = sqlite3.connect(DB_FILE)
        conn.row_factory = sqlite3.Row
        c = conn.cursor()
        c.execute('SELECT * FROM soil_data ORDER BY id DESC')
        rows = c.fetchall()
        conn.close()
        return jsonify([dict(row) for row in rows]), 200
    except Exception as e:
        return jsonify({'error': str(e)}), 500
@app.route('/soil', methods=['POST'])
def add_soil_data():
    try:
        data = request.get_json()
        conn = sqlite3.connect(DB_FILE)
        c = conn.cursor()
        c.execute('''
            INSERT INTO soil_data (temperature, humidity, rainfall, soil_type, recommended_crop)
            VALUES (?, ?, ?, ?, ?)
        ''', (
            float(data['temperature']), float(data['humidity']),
            float(data['rainfall']), str(data['soil_type']),
```

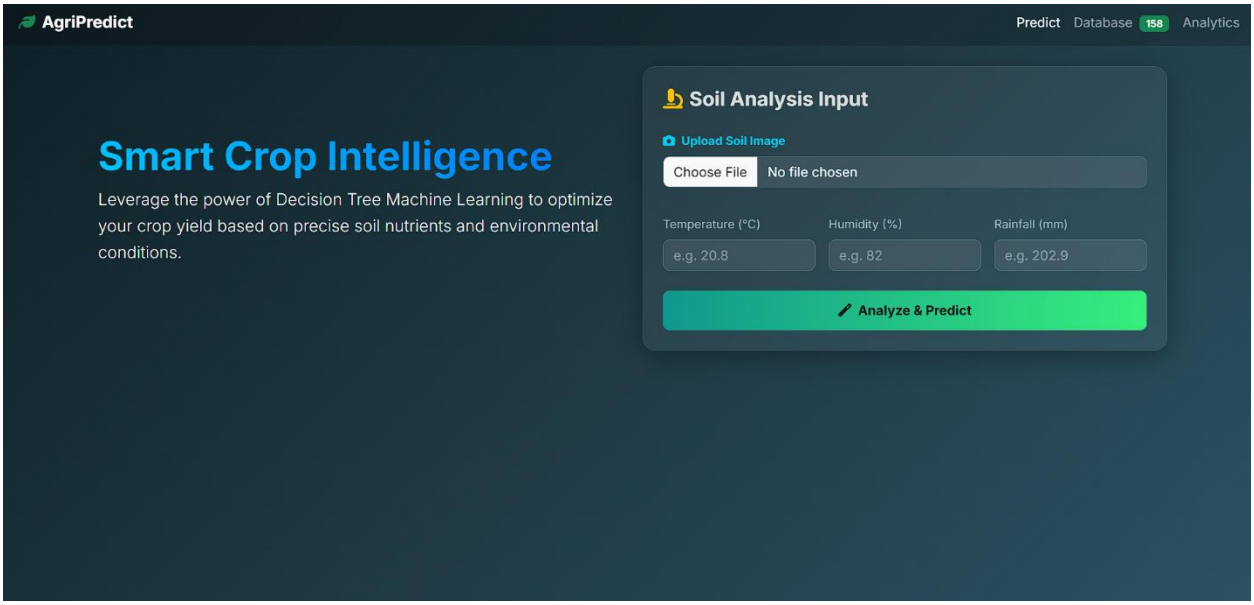
```
            str(data['recommended_crop'])
        ))
        conn.commit()
        inserted_id = c.lastrowid
        conn.close()
        return jsonify({'message': 'Record added', 'id': inserted_id}), 201
    except Exception as e:
        return jsonify({'error': str(e)}), 500
@app.route('/soil/<int:id>', methods=['PUT'])
def update_soil_data(id):
    try:
        data = request.get_json()
        fields, values = [], []
        for key in ['temperature', 'humidity', 'rainfall', 'soil_type', 'recommended_crop']:
            if key in data:
                fields.append(f"{key} = ?")
                values.append(data[key])
        if not fields: return jsonify({'error': 'No fields provided'}), 400
        values.append(id)
        conn = sqlite3.connect(DB_FILE)
        c = conn.cursor()
        c.execute(f"UPDATE soil_data SET {', '.join(fields)} WHERE id = ?", tuple(values))
        conn.commit()
```

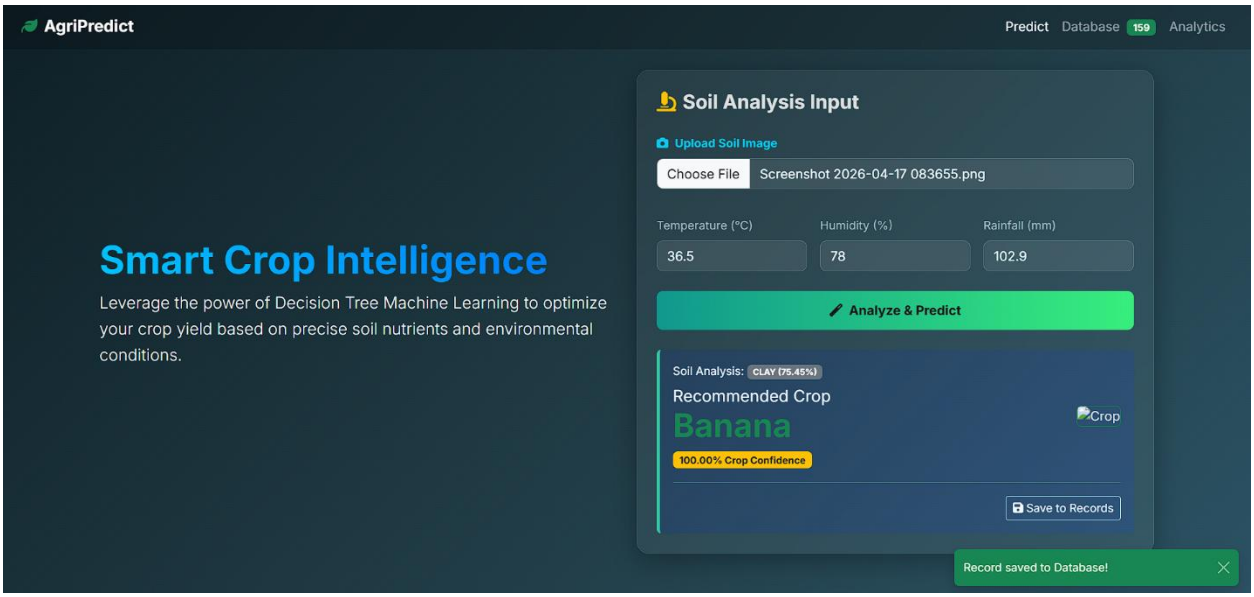
```
        conn.close()
        return jsonify({'message': 'Updated successfully'}), 200
    except Exception as e:
        return jsonify({'error': str(e)}), 500
@app.route('/soil/<int:id>', methods=['DELETE'])
def delete_soil_data(id):
    try:
        conn = sqlite3.connect(DB_FILE)
        c = conn.cursor()
        c.execute('DELETE FROM soil_data WHERE id = ?', (id,))
        conn.commit()
        conn.close()
        return jsonify({'message': 'Deleted successfully'}), 200
    except Exception as e:
        return jsonify({'error': str(e)}), 500
@app.route('/analytics', methods=['GET'])
def get_analytics():
    try:
        conn = sqlite3.connect(DB_FILE)
        conn.row_factory = sqlite3.Row
        c = conn.cursor()
        c.execute('SELECT recommended_crop, COUNT(*) as count FROM soil_data GROUP BY
recommended_crop')
        crop_dist = {row['recommended_crop']: row['count'] for row in c.fetchall()}
```

```
        c.execute('SELECT soil_type, COUNT(*) as count FROM soil_data GROUP BY
soil_type')

        soil_dist = {row['soil_type']: row['count'] for row in c.fetchall()}

        c.execute('SELECT AVG(temperature) as avg_T, AVG(humidity) as avg_H, AVG(rainfall)
as avg_R FROM soil_data')

        avg_row = c.fetchone()

        averages = dict(avg_row) if avg_row else {'avg_T': 0, 'avg_H': 0, 'avg_R': 0}

        conn.close()

        return jsonify({

            'crop_distribution': crop_dist,

            'soil_distribution': soil_dist,

            'averages': averages,

            'total_records': sum(crop_dist.values())

        }), 200

    except Exception as e:

        return jsonify({'error': str(e)}), 500

if __name__ == '__main__':

    app.run(debug=True, port=5000)
```

OUTPUT

Screenshots of the Model





DataBase of the Crops And Soil Records

AgriPredict

Predict Database 159 Analytics

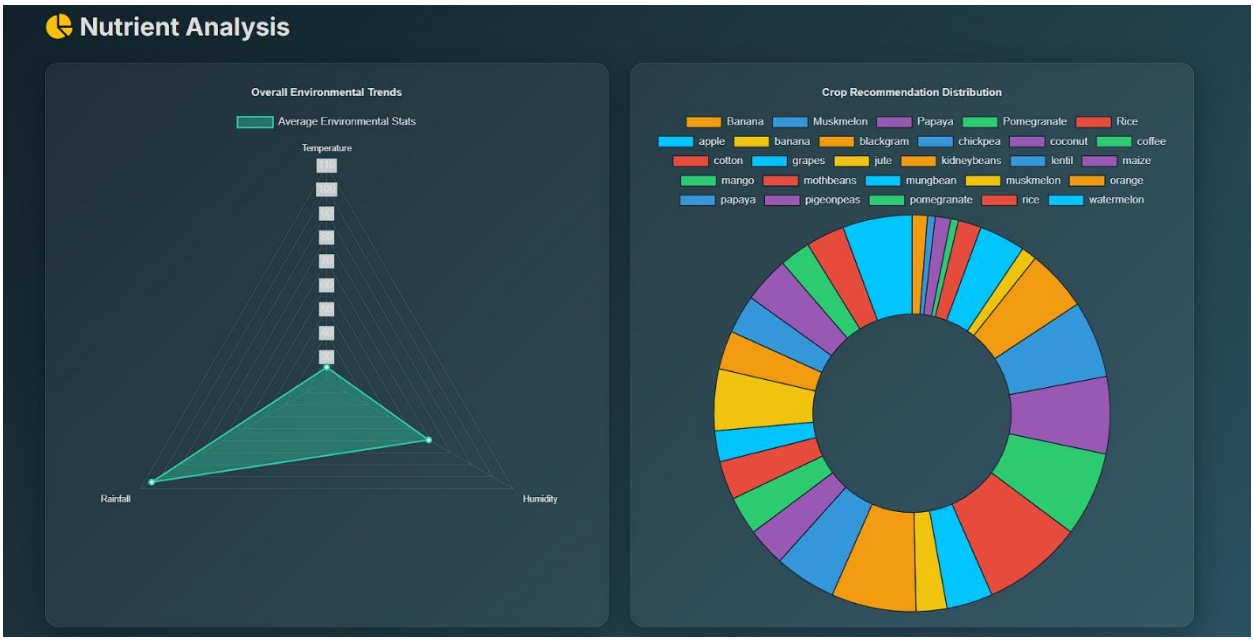
Soil Records

Refresh

ID	Temp	Humid	Rain	Soil Type	Crop	Actions
#159	36.5	78.0	102.9	CLAY	Banana	
#158	38.0	82.0	202.0	CLAY	Rice	
#157	38.0	82.0	202.0	SANDY	Rice	
#156	38.0	78.0	100.0	CLAY	Banana	
#155	43.0	90.0	202.0	CLAY	Papaya	
#154	23.0	90.0	202.0	CLAY	Papaya	
#153	27.8	86.0	101.0	BLACK	Pomegranate	
#152	28.0	79.0	0.0	ALLUVIAL	Muskmelon	
#151	27.6	84.0	208.0	CLAY	Rice	
#150	36.2	59.5	128.9	ALLUVIAL	pigeonpeas	
#149	30.4	66.7	72.3	BLACK	blackgram	

#150	36.2	59.5	128.9	ALLUVIAL	pigeonpeas	
#149	30.4	66.7	72.3	BLACK	blackgram	
#148	26.2	64.9	182.6	SANDY	pigeonpeas	
#147	20.8	21.2	114.7	ALLUVIAL	kidneybeans	
#146	24.4	60.3	44.5	ALLUVIAL	lentil	
#145	26.5	94.1	201.5	ALLUVIAL	papaya	
#144	25.3	82.2	104.3	ALLUVIAL	banana	
#143	26.0	94.4	136.5	ALLUVIAL	coconut	
#142	25.7	83.3	72.9	CLAY	cotton	
#141	20.4	91.6	112.1	ALLUVIAL	orange	
#140	27.2	95.2	159.5	SANDY	coconut	
#139	19.6	20.3	137.7	RED	kidneybeans	
#138	26.0	31.7	171.4	SANDY	pigeonpeas	
#137	25.3	96.9	173.6	ALLUVIAL	coconut	
#136	21.4	60.4	92.2	ALLUVIAL	maize	

Analysis



Performance Metrics:

$$Accuracy = \frac{TP + TN}{Total}$$

$$Precision = \frac{TP}{TP + FP}$$

$$Recall = \frac{TP}{TP + FN}$$

$$F1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}$$

System Displays:

- Soil Type
- Recommended Crop
- Confidence Score
- Crop Image

RESULTS AND FUTURE ENHANCEMENT

The developed system successfully recommends suitable crops based on soil images and environmental conditions. The hybrid approach using Deep Learning (MLP) for soil classification and Decision Tree for crop prediction provides reliable results.

- The model achieved high accuracy (around 98%) on the test dataset
- The system provides real-time predictions with confidence levels
- It eliminates the need for prior agricultural knowledge
- The web application is user-friendly and efficient

Overall, the system demonstrates that combining image-based analysis with machine learning improves decision-making in agriculture.

- Replace MLP with CNN for better image classification accuracy
- Integrate real-time weather APIs to auto-fetch environmental data
- Use real-world agricultural datasets for improved reliability
- Develop a mobile application for wider accessibility
- Add multilingual support for farmers in different regions

Git Hub Link of the project along with the report	https://github.com/BARATH0202/Crop-Recommendation-System.git

REFERENCES

1. Tom M. Mitchell,
   Machine Learning, McGraw-Hill Education, 1997.
2. IEEE,
   "Crop Prediction based on Environmental Factors using Machine Learning," IEEE Conference Papers.
3. Aurélien Géron,
   Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow, O'Reilly Media, 2019.
4. Scikit-learn Documentation,
   Decision Tree Classifier
   https://scikit-learn.org/stable/modules/tree.html
5. Kaggle Dataset Repository,
   Crop Recommendation Dataset
   https://www.kaggle.com/datasets